

Iterative Feature Engineering and Validation-Driven Model Selection for NCAA Tournament Outcome Prediction

An R&D Exercise Conducted During a LeverX Data Science Internship

Richard Stoiberer

LeverX Internship Program

July 2026

Abstract

This report documents a self-directed research-and-development exercise carried out during a LeverX data science internship. A public Kaggle competition (March Machine Learning Mania 2026) was used as a fully self-contained testbed for a validation-first approach to sports outcome prediction: rather than reaching for the most sophisticated available model, a gender-split logistic regression was implemented entirely from first principles (Newton's method, no scikit-learn) and built up through nineteen individually validated notebooks, each testing one specific hypothesis against a strict walk-forward validation scheme. No proprietary LeverX data or client information was used at any point; the dataset is historical public sports data. A third of the ideas tested improved the validated score and were kept — separating men's and women's models, adding a margin-of-victory-weighted Elo rating, a conference-strength feature, and an unsupervised team-clustering signal — while the rest, including detailed box-score statistics, tree-based gradient-boosted models, recent-form momentum features, feature interaction terms, and a widened training window, were tested with equal rigor and discarded. The model was submitted twice to the live competition leaderboard. The central contribution of this exercise is not the specific score achieved but a concrete, honestly documented demonstration of how validation discipline, rather than model sophistication, drove every decision made.

1. Introduction

This project began as a self-directed exercise within a LeverX internship, with two goals in mind. First, to produce a complete, start-to-finish example of an iterative, validation-driven feature engineering workflow — from exploratory data analysis through nineteen progressively refined modeling notebooks to two real competition submissions — that could later serve as a template for similar structured-data problems. Second, to treat core statistical and machine learning methods as something to build and understand from first principles rather than as black boxes: the logistic regression model used throughout, the distributional tests used to make a data-scope decision, and the clustering and dimensionality reduction used in the later exploratory work were all implemented by hand, using only numpy and pandas.

The vehicle for this exercise was Kaggle's March Machine Learning Mania 2026 — a public competition predicting the outcomes of the 2026 NCAA Division I men's and women's basketball tournaments, with no connection to LeverX client data, proprietary systems, or confidential information of any kind. It was chosen because it combines a well-defined, rigorously scoreable task with a genuinely limited amount of labeled training data, a combination that rewards careful validation methodology far more than raw model complexity.

2. Problem and Data

The task is to predict, for every possible pairing of Division I teams, the probability that the team with the lower official Team ID beats the team with the higher one, separately for the men's and women's tournaments. The competition is scored on the Brier score: the mean squared error between the predicted probability and the actual 0/1 outcome, where lower is better. The submission file itself covers all 132,133 possible team pairings across both tournaments — far more than the 68-team bracket each gender actually seeds, since the file format is fixed before Selection Sunday. In practice, only pairings between two teams that made that year's tournament are ever scored.

The underlying data spans 35 CSV files covering men's results back to 1985 and women's results back to 1998, including box scores, tournament seeds, conference membership, and (men's only) third-party power rankings. Two data characteristics shaped every modeling decision that followed. First, the 2026 tournament itself has no historical outcome file to train or validate against — the only way to obtain a real, ground-truth score is a live submission to the competition's leaderboard. Second, the two natural sources of labeled data differ by nearly two orders of magnitude: only 4,302 historical tournament games exist to train and validate a model against the exact distribution being predicted, against 341,084 regular-season games that match a systematically different context (home-court advantage, blowouts, non-tournament teams). Figure 1 makes this gap concrete. The resolution adopted throughout this project uses regular-season data only to compute team-strength features, while training and validating the classifier itself exclusively on tournament games, keeping the label distribution aligned with the real prediction target.

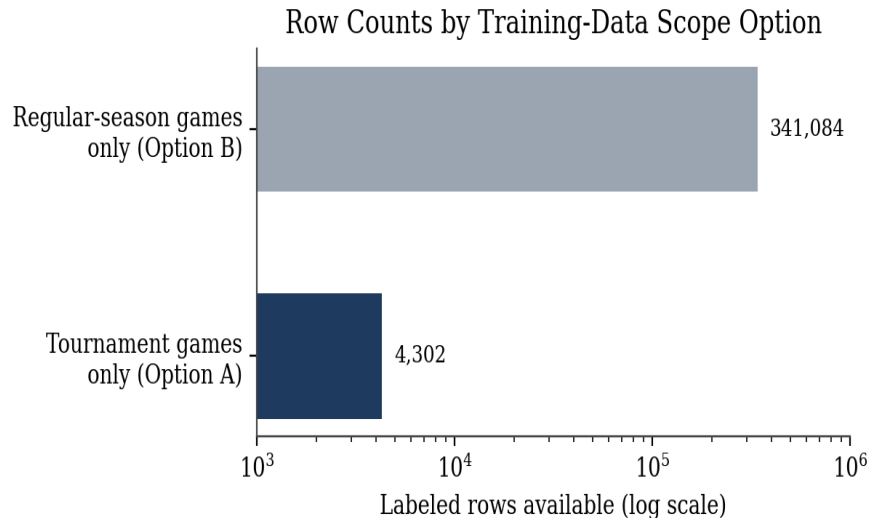


Figure 1. Row counts available under two candidate training-data scopes. The wide gap motivated a hybrid approach: regular-season data builds the features, but only tournament games are used as training labels.

3. Methodology

3.1 Exploratory Analysis and a Data-Scope Decision

Initial analysis went table by table through the raw data to check row counts, missingness, season coverage, and ID consistency before any joining began. This surfaced the 2026 ground-truth gap and the row-count asymmetry above, along with a genuine structural difference between the two tournaments: the women's field expanded from 64 to 68 teams in 2022, and women's detailed box-score statistics only begin in 2010 versus

2003 for men's, a shorter usable history for any feature built from those columns.

One specific question — whether training data should widen from tournament-only games to include the regular season — was answered with a formal distributional comparison rather than intuition: a hand-implemented Levene's test, Welch's t-test, and two-sample Kolmogorov-Smirnov test compared the winning-margin distribution of regular-season and tournament games, separately for each gender. The tests found the two distributions distinct enough, for the men's data specifically, that widening the training window was judged safe; for the women's data the tournament-only distribution stood far enough apart that widening was not adopted. Both decisions were tested empirically in later notebooks and the widened option was ultimately rejected once real validation numbers came back (Section 5.1).

3.2 Baseline Model

Before settling on this approach, three alternative directions were tried and quickly ruled out under the same walk-forward harness. Adding detailed box-score statistics on top of a compact three-feature baseline scored 0.1759, worse than the baseline itself. Replacing that compact feature set with seed-only or power-ranking-only models, including a third-party Massey Ordinal ranking, scored substantially worse still, 0.2031–0.2063. Gradient-boosted tree models (LightGBM and XGBoost), tried against both the compact and detailed feature sets, scored 0.1775–0.1822 — also worse than a simple logistic regression. None of the three directions were pursued further; the remainder of this report follows the logistic-regression line established next.

A first model was built quickly and deliberately kept simple: a hand-written logistic regression, fit via Newton's method (iteratively reweighted least squares) with a light L2 penalty, using three features — seed difference, win-percentage difference, and average-scoring-margin difference between the two teams. No machine learning library was used for the model itself; the only external dependencies were numpy and pandas. The evaluation harness used throughout the entire project, shown in Listing 1, is a walk-forward validation: for each of five held-out seasons (2021 through 2025), the model trains only on strictly prior seasons and is scored on the held-out one, mirroring the real prediction task of forecasting a season with no future information available.

Listing 1. The walk-forward validation harness reused, unchanged, by every one of the nineteen notebooks in this project.

```
FOLD_SEASONS = [2021, 2022, 2023, 2024, 2025]
for fold_season in FOLD_SEASONS:
    # train strictly on seasons before the held-out one -- never
    # let the model see a season it is about to be scored on
    train = matchups[matchups["Season"] < fold_season]
    test = matchups[matchups["Season"] == fold_season]
    w = fit_logreg(train[FEATURES].values, train["Label"].values)
    brier = brier_score(predict_proba(test[FEATURES].values, w), test["Label"].values)
```

This baseline, trained on all history pooled across both genders, reached a walk-forward average Brier score of 0.1739. Every subsequent notebook was judged against whatever the current best score was at the time, never against this original baseline alone.

3.3 Iterative Development

From this baseline, development proceeded through a sequence of individually validated notebooks, each isolating exactly one change: splitting into separate men's and women's models; widening the men's training window per the data-scope test above; adding recent-form ("momentum") features; adding a

margin-of-victory-weighted Elo rating; extending that Elo system with conference-tournament games; adding a conference-strength feature; tuning the Elo K-factor per gender; adding feature interaction terms; and finally, testing whether unsupervised team clustering could contribute a useful signal. Each stage's outcome, adopted or rejected, is detailed in Sections 4 and 5.

4. Results

Figure 2 summarizes walk-forward Brier score across the major milestones of the project, with diamonds marking the two points where a model was actually submitted to the live Kaggle leaderboard. Table 1 presents the same milestones numerically, alongside every notable idea that was tested and rejected along the way.

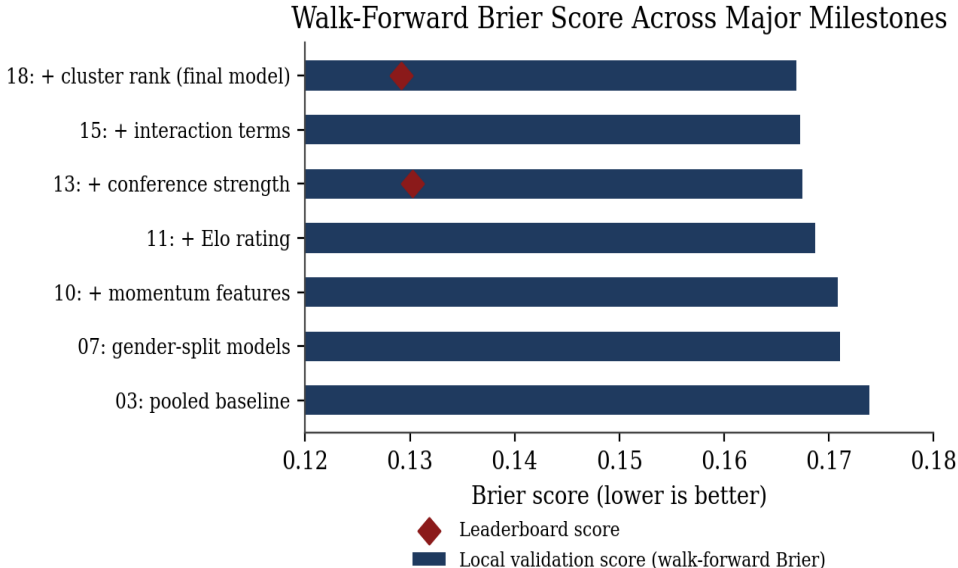


Figure 2. Walk-forward Brier score across major milestones (bars; lower is better) with the two real leaderboard submissions marked as diamonds.

Notebook	Approach	Features	Local (Brier)	Leaderboard
03	Pooled baseline	3	0.1739	—
07	Gender-split models	3	0.1711	—
10	+ recent-form (momentum) features	5	0.1709	—
11	+ margin-of-victory Elo rating	4	0.1687	—
13	+ conference strength	5	0.1675	0.1302747
14	Elo K-tuning + conference-strength bug fix	5	0.1683–0.1690	—
15	+ feature interaction terms	7	0.1673	—
18 / 19	+ cluster rank (final model)	6	0.1669	0.1292044

Table 1. Walk-forward Brier score by approach. Notebooks 14 (K-tuning) and the plain gender-split-only and momentum rows show ideas that were tested and not adopted; the final model builds on notebook 13, not 14 or 15 (see Sections 5.1–5.2).

5. Key Findings

5.1 Feature Engineering Showed Diminishing Returns Once Core Signals Were Captured

Several plausible-sounding ideas were tested with the same rigor as the ones that worked, and did not hold up. Widening the men's training window to include regular-season games (motivated by the distributional test in Section 3.1) scored 0.1715, slightly worse than the tournament-only gender split. Adding each team's win percentage and scoring margin over their last ten games as a "momentum" signal produced an essentially flat 0.1709, indistinguishable from noise. Two hand-built interaction terms (seed difference multiplied by Elo difference, and by conference strength) moved the score by only 0.0001–0.0002, and a check confirmed the interaction terms were genuinely uncorrelated with the base features, they simply did not carry enough independent signal to matter. In every one of these cases, a plausible hypothesis and a properly implemented feature were not, by themselves, sufficient — only the held-out walk-forward score decided whether an idea survived.

5.2 An Implementation Bug That Improved the Validated Score

The conference-strength feature (notebook 13) computes each team's conference average Elo rating by grouping teams on season and conference abbreviation. While building a later notebook, a real defect was discovered in this calculation: the men's and women's conference-membership files use the same conference abbreviation strings (for example, the same code for the Big Ten in both files), so the grouping silently blended men's and women's teams together when computing a conference's average strength, shown in Listing 2.

Listing 2. The single line responsible for the cross-gender data-blending defect in the conference-strength feature.

```
# MTeamConferences.csv and WTeamConferences.csv share ConfAbbrev strings,  
# so this groupby silently blends men's and women's teams together:  
conf_strength = conf_teams.groupby(["Season", "ConfAbbrev"])["Elo"].mean()
```

A corrected version, grouping separately by gender, was built and evaluated under the exact same walk-forward harness. It scored 0.1683, worse than the original, uncorrected 0.1675. Given this project's own standing rule — that held-out validation performance, not code cleanliness, decides what gets kept — the original, buggy version remained the project's best model, and the corrected version was not adopted. This is reported here exactly as it occurred: a genuine defect, caught, measured, and knowingly kept because fixing it made the model perform worse against the same rigorous evaluation used for every other decision in this project. It is flagged explicitly rather than silently corrected, since a future maintainer of this code should be aware the defect exists and was a deliberate, evidence-based call rather than an oversight.

5.3 Unsupervised Clustering Surfaced Real Structure, With Modest Predictive Value

Separately from the supervised prediction task, a hand-written k-means clustering (with an elbow-method choice of six clusters) was run on team-season profiles — seed, win percentage, scoring margin, Elo rating, and conference strength — to see whether meaningful tiers would emerge without being told the tournament outcome. They did: Figure 3, a hand-written principal component projection capturing 91.7% of the variance in two dimensions, shows six visually distinct groups that line up with recognizable basketball tiers, from an elite cluster of mostly 1-4 seeds down to a clear underdog tier. One individual finding stood out: a 15-seed in the 2026 field landed in a "mid-major overachiever" tier rather than the underdog tier almost every other 15/16 seed falls into, driven by an unusually strong win percentage and scoring margin against a weak conference

schedule.

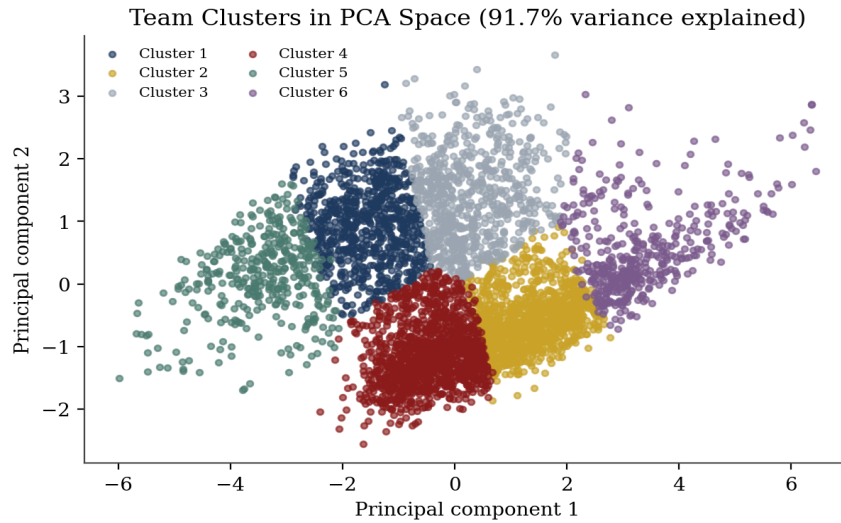


Figure 3. Team-season clusters projected into two principal components (91.7% of variance retained). Clusters were then re-fit separately by gender and ranked by average Elo to produce a predictive feature.

Turning this into an actual prediction required care, and the elbow method itself was re-run separately for each gender rather than reused from the pooled exploration above, landing on five clusters for the men's field and eight for the women's. Predicting a matchup from cluster membership alone, with no other information, scored 0.1886 — considerably worse than the existing model, confirming that reducing a team down to one of five or eight buckets throws away real precision. Adding each team's cluster rank as one additional feature alongside the existing five, however, produced a genuine, if modest, improvement to 0.1669, becoming the project's final model.

5.4 Validation Estimates and the Live Leaderboard Diverged, Favorably

Both real submissions to the Kaggle leaderboard scored noticeably better than their own walk-forward validation estimate had predicted, shown in Table 2. Rather than a sign the model is secretly stronger than measured, this is best explained by sample size: the 2026 tournament actually scored on the order of 126 total games, a favorite-heavy ("chalky") year by other competitors' independent accounts, and a small, single sample of games can swing an observed score substantially in either direction relative to a five-season validation average. The fact that both submissions, built from different feature sets and five days apart, show a gap of a very similar size (0.0372 and 0.0377) is itself informative: it points toward a systematic property of this particular tournament's outcomes rather than a one-off fluke in a single submission.

Submission	Local (walk-forward)	Leaderboard	Gap (local – LB)
1 (notebook 16, conference-strength model)	0.1675	0.1302747	+0.0372
2 (notebook 19, cluster-rank model)	0.1669	0.1292044	+0.0377

Table 2. Local walk-forward estimate versus actual Kaggle leaderboard score for both real submissions. A positive gap means the leaderboard score was better (lower Brier) than the local estimate.

The walk-forward number remains the more honest, repeatable estimate of this model's typical performance across many possible tournaments; the leaderboard score reflects how it did on the one tournament that

actually counted this year.

6. Deployment: From Notebook to Live Prediction Service

6.1 Motivation and Architecture

The final model existed, up to this point, only as a notebook that produced a static CSV file. To make it usable outside that context — callable directly by another party, such as a fellow intern working the same competition, rather than re-run from source — the trained model was exported and wrapped in a small HTTP service. The web layer (FastAPI) was deliberately kept separate from the prediction logic itself: a single module loads the exported model once at startup and exposes one pure function, `predict_matchup`, which the API layer calls but does not otherwise depend on. The service was packaged as a Docker container — a fixed, minimal Python 3.11 image carrying only the two runtime dependencies, FastAPI and Uvicorn — so it runs identically regardless of where it is deployed, and hosted on Railway, which builds and runs that container directly from the project's GitHub repository and provides a stable public HTTPS endpoint without separate server administration. A single POST `/predict` endpoint accepts a batch of up to 128 matchups per request, a cap matching guidance given for this exercise, and returns one result per matchup rather than requiring one request per prediction.

6.2 Correctness Guarantees Carried Over from the Model

Two properties of the underlying model needed explicit handling to survive the transition from notebook to service. First, the model was trained with the lower-numbered team always in the Team1 position; the service evaluates every request internally in that same canonical direction and returns either that probability or its complement, guaranteeing $P(A \text{ beats } B) + P(B \text{ beats } A) = 1$ exactly, regardless of which order a caller happens to supply the two teams in (Listing 3). Second, three conditions are rejected per-row rather than allowed to fail an entire batch: a team ID outside the 2026 field, a men's team paired against a women's team, and a team paired against itself — each returns a descriptive error alongside the offending matchup while the remaining rows in the same batch are still scored normally.

Listing 3. Requesting the same matchup in reversed order returns 0.2070 — exactly $1 - 0.7930$, the probability returned for the original ordering — confirming the canonical-ordering guarantee holds through the live API, not only inside the notebook.

```
POST /predict
{"matchups": [{"team1": 1163, "team2": 1181}]}

{"predictions": [{"team1": 1163, "team2": 1181,
                  "prediction": 0.2070, "error": null}]}
```

6.3 Verification Against the Submitted Model

Deploying a model is not the same as deploying the validated model, so the live service was checked against the file already scored on the competition leaderboard. A script reconstructed the full 132,133-row submission by calling the live endpoint, in the same 128-matchup batches the server itself enforces, for exactly the 4,556 rows representing real 2026-tournament pairings; the remaining rows, involving at least one non-tournament team, were left at the submission format's default of 0.5, matching the original notebook's own convention. Compared row-by-row against the file that scored leaderboard rank 482, the reconstructed file matched to a maximum difference of 1.8×10^{-6} and a mean difference of 1.2×10^{-8} , fully attributable to the exported model weights being serialized at 8 significant digits rather than full floating-point precision, and far too small to

move a Brier score at any reportable digit. This confirms the deployed service is not a separate reimplement that merely resembles the validated model, but a faithful, verified copy of the exact artifact that was scored.

7. Reflection: Validation Discipline as the Real Deliverable

Of the twelve distinct hypotheses tested across nineteen notebooks, four improved the validated score and were kept; the rest were rejected. This ratio is not a disappointing outcome, it is the expected and honest shape of feature engineering work done properly. A model that improved on every single idea tried would be a strong signal that the validation methodology itself was not trustworthy, not that every idea was equally good.

The clearest practical lesson from this exercise concerns small-sample evaluation. A single competition outcome, or any single held-out test with on the order of a hundred scored events, is not by itself proof of a model's quality in either direction: it did not, in this case, prove the model was better than five seasons of validation suggested, nor would a bad single result have proven the model was worse. The five-season walk-forward average is the more defensible number to trust going into a decision; the live result is simply what happened once. Recognizing which of these two numbers a given decision should actually be based on is a skill this exercise helped make concrete, and applies well beyond sports prediction to any small-sample evaluation setting.

8. Recommendations for LeverX

8.1 Methodology Assessment

Walk-forward validation (train strictly on prior periods, validate on a later held-out one) is recommended over a random train/validation split for any problem with a time or sequence component, since a random split can let a model implicitly learn from information that would not have been available at prediction time. Correlation checks before trusting any new feature proved genuinely useful more than once in this project, including one case (Section 5.1) where a feature was confirmed uncorrelated with existing ones yet still failed to improve the score, a reminder that being uncorrelated is not the same as being useful.

8.2 Process Template

The overall workflow developed here — one notebook per hypothesis, numbered in build order, each ending in a specific, numbers-cited verdict rather than a vague summary — is intended as a reusable template for future practice rounds or early-stage prototyping on similar structured-data problems. The accompanying repository is organized with that reuse in mind: a short top-level README, a detailed working-notes document, and every notebook, including the ones whose ideas were rejected, kept rather than deleted.

8.3 Practical Notes Worth Carrying Forward

Two items are worth explicit institutional memory. First, the conference-strength defect documented in Section 5.2: shared identifier strings across otherwise-separate data files are an easy source of silent, hard-to-notice data leakage between groups that should be considered on any project joining multiple related datasets. Second, the validation-versus-leaderboard gap in Section 5.4: a single live result, in either direction, should be weighed against a proper held-out validation estimate rather than taken at face value, particularly on any task with a genuinely small number of scored outcomes.

9. Conclusion

This exercise produced a complete, gender-split tournament prediction pipeline built entirely from first principles; a disciplined, walk-forward-validated evaluation of twelve distinct modeling hypotheses; and two real submissions to a live competition leaderboard, the second improving on the first. Beyond the specific score achieved, it produced several concrete, transferable findings: on the diminishing returns of feature engineering once core signals are captured, on how a data-integrity defect can be caught, measured, and honestly reported rather than silently fixed or silently ignored, and on the practical risk of over-interpreting any single small-sample result. Perhaps most usefully, it offered a first-hand, concrete illustration of how validation discipline, consistently applied, is what actually separates a defensible modeling decision from a plausible-sounding one.

Note: the dataset used throughout this report is a public Kaggle competition dataset (March Machine Learning Mania 2026, historical NCAA basketball results). No LeverX client data, proprietary systems, or confidential information were used at any point in this exercise.